

DisrtuCare — La communication Logiciel ↔ Matériel en détail

Document technique de référence : protocole, API, séquences et code

Projet DisrtuCare

Juin 2026

Table des matières

1	Introduction et périmètre	3
2	Architecture générale	4
2.1	Le schéma physique et logique	4
2.2	Les rôles : qui est serveur, qui est client	4
2.3	Pourquoi ce choix de transport	5
3	Le protocole : les sept points d'accès	6
4	Lecture de l'état : l'endpoint /status	8
4.1	Ce que renvoie l'appareil	8
4.2	Le code qui la produit	9
5	Le modèle d'événements : « pousser » sans push	10
5.1	Le problème et sa solution	10
5.2	Pourquoi un compteur plutôt qu'une file	10
5.3	Les types d'événements	11
6	Écriture vers l'appareil : les routes POST	12
6.1	/sync — pousser l'horaire et le médicament	12
6.2	/settime — régler l'horloge	13
6.3	/motor — faire tourner le distributeur	13
6.4	/led — piloter la LED indicatrice	14
6.5	/diag et /selftest — diagnostics	14
7	Le client HTTP côté application	15
7.1	La gestion de l'adresse	15
7.2	Le cycle de connexion, étape par étape	15
7.3	La boucle d'interrogation	16
7.4	L'envoi des commandes	17
8	Scénarios de bout en bout	18
8.1	Scénario A — la dose programmée	18
8.2	Scénario B — la première connexion	19
8.3	Scénario C — la modification de l'horaire	19
8.4	Scénario D — la perte de réseau	19
9	Robustesse et détails d'implémentation	20

10 Référence rapide	21
10.1 Tableau récapitulatif des échanges	21
10.2 Constantes clés	21
10.3 Conclusion	22

Chapitre 1

Introduction et périmètre

Ce document décrit, dans le détail et sans rien supposer d'acquis, **la manière dont l'application mobile DisrtuCare et le distributeur de médicaments communiquent**. Il est conçu pour être lu d'un bout à l'autre : chaque mécanisme est d'abord expliqué en mots, puis illustré par le code réellement exécuté des deux côtés du lien.

Le système se compose de deux éléments qui dialoguent :

- **L'application mobile** — écrite en React Native (TypeScript) et exécutée dans Expo Go sur un téléphone Android. Elle ne contient aucune logique matérielle de bas niveau : elle envoie des requêtes et lit des réponses.
- **Le distributeur** — une carte **NodeMCU V3 (puce ESP8266)** qui pilote un écran LCD, un moteur pas-à-pas et un bouton. Elle héberge un petit **serveur web** et contient toute la logique temps réel (distribution, confirmation, persistance).

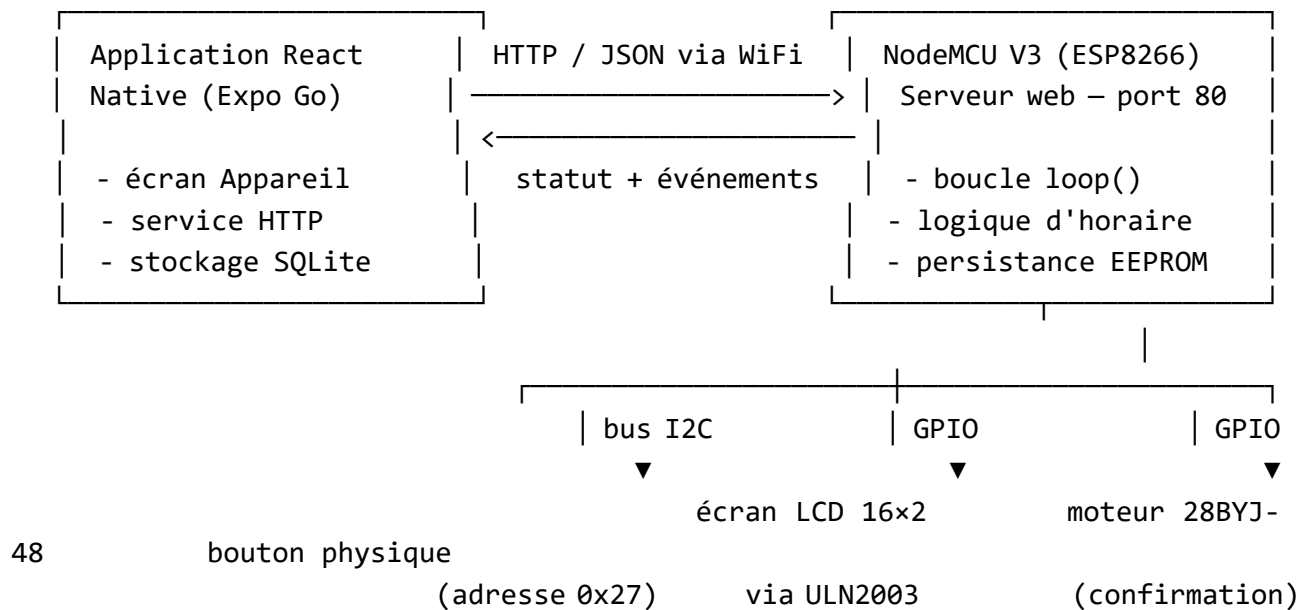
Le fil conducteur de tout le document est simple : **le téléphone configure et observe ; l'appareil agit de façon autonome ; l'état est réconcilié par interrogation périodique**. Tout le reste découle de ce principe.

À retenir d'emblée. Il n'y a ni Bluetooth, ni module natif, ni cloud, ni base de données distante. La communication est uniquement du **HTTP transportant du JSON**, échangé sur le **réseau WiFi local** auquel le téléphone et l'appareil sont tous deux connectés.

Chapitre 2

Architecture générale

2.1 Le schéma physique et logique



Le téléphone et la carte doivent être sur **le même sous-réseau WiFi**. L'application ne « découvre » pas l'appareil automatiquement : l'utilisateur lit l'adresse IP affichée sur l'écran LCD au démarrage et la saisit dans l'application. Cette adresse est ensuite mémorisée.

2.2 Les rôles : qui est serveur, qui est client

C'est le point le plus important à intégrer, car il oriente tout le protocole :

- **L'appareil est le serveur.** Il écoute en permanence sur le port 80 et attend qu'on l'interroge. Il ne prend jamais l'initiative d'ouvrir une connexion vers le téléphone.

- **L'application est le client.** C'est toujours elle qui initie chaque échange, qu'il s'agisse d'envoyer un ordre ou de lire l'état.

Conséquence directe : l'appareil **ne peut pas notifier spontanément** l'application quand quelque chose se produit (par exemple lorsqu'une pilule est distribuée). Le problème est résolu par **l'interrogation périodique** (« polling ») décrite plus loin : l'application redemande régulièrement l'état, et détecte les changements.

2.3 Pourquoi ce choix de transport

Le choix de HTTP/JSON sur WiFi n'est pas arbitraire ; il a des conséquences concrètes :

Décision	Pourquoi	Conséquence pratique
HTTP/JSON sur le port 80	Universel et lisible	Testable depuis n'importe quel navigateur, sans aucun SDK installé sur le téléphone
Appareil = serveur	L'ESP8266 a une IP fixe sur le LAN	L'app trouve l'appareil par son adresse IP, jamais l'inverse
REST sans état + polling	Simplicité et robustesse	Aucun socket permanent à maintenir vivant ou à reconnecter
fetch() standard	Compatibilité Expo Go	Aucun module natif → l'app tourne sans build natif
CORS activé (*)	Tolérance des clients web	Fonctionne aussi depuis un navigateur de bureau pour le débogage
Horaire en EEPROM	Autonomie	L'appareil conserve sa configuration même hors tension

Chapitre 3

Le protocole : les sept points d'accès

L'appareil expose **sept routes HTTP**. Elles se répartissent en deux familles : les lectures (GET) et les écritures (POST). Voici la vue d'ensemble, suivie du détail de chacune.

Endpoint	Méthode	Famille	Rôle résumé
/status	GET	Lecture	Renvoie l'horaire, l'heure et le dernier événement
/sync	POST	Écriture	Met à jour les heures AM/PM et le nom du médicament
/settime	POST	Écriture	Règle l'horloge interne de l'appareil
/led	POST	Écriture	Allume ou éteint la LED indicatrice
/motor	POST	Écriture	Fait tourner le moteur de N crans (test manuel)
/diag	GET	Lecture	Diagnostic matériel (I2C, RTC, mémoire, signal WiFi)
/selftest	GET	Lecture	Lance un test du moteur pas-à-pas à la demande

Les routes sont enregistrées une fois pour toutes au démarrage, dans `setup()` :

```
server.on("/status", HTTP_GET, handleStatus);  
server.on("/sync", HTTP_POST, handleSync);
```

```
server.on("/settime", HTTP_POST, handleSetTime);
server.on("/led",      HTTP_POST, handleLed);
server.on("/motor",    HTTP_POST, handleMotor);
server.on("/diag",     HTTP_GET,  handleDiag);
server.on("/selftest", HTTP_GET,  handleSelfTest);
server.onNotFound([]() { cors(); server.send(404, "application/json",
                                                "{\"error\":\"not found\"}"); });
server.begin();
```

Toute route inconnue renvoie un 404 avec un corps JSON explicite. Chaque réponse porte les en-têtes CORS (voir §6.3).

Chapitre 4

Lecture de l'état : l'endpoint /status

4.1 Ce que renvoie l'appareil

/status est la route centrale. C'est par elle que l'application apprend tout ce qui se passe sur l'appareil. Une réponse typique ressemble à ceci :

```
{
  "am": "08:00",
  "pm": "20:00",
  "med": "Metformin",
  "time": "14:30",
  "rtc": true,
  "awaiting": false,
  "event_id": 3,
  "last_event": { "evt": "confirmed", "type": "AM" }
}
```

Signification champ par champ :

Champ	Type	Signification
am	chaîne HH:MM	Heure de la dose du matin
pm	chaîne HH:MM	Heure de la dose du soir
med	chaîne	Nom du médicament configuré
time	chaîne HH:MM	Heure courante de l'horloge de l'appareil
rtc	booléen	true si l'horloge tourne (heure valide)
awaiting	booléen	true si une dose a été distribuée mais pas encore confirmée

Champ	Type	Signification
event_id	entier	Compteur monotone, incrémenté à chaque événement
last_event	objet ou null	Le dernier événement survenu

4.2 Le code qui la produit

Côté appareil, la réponse est assemblée à la main pour éviter une double sérialisation de `last_event` (qui est déjà du JSON) :

```
static void handleStatus() {
    cors();
    char amStr[6], pmStr[6];
    snprintf(amStr, sizeof(amStr), "%02d:%02d", amHour, amMin);
    snprintf(pmStr, sizeof(pmStr), "%02d:%02d", pmHour, pmMin);

    char buf[320];
    snprintf(buf, sizeof(buf),
        "{\"am\":\"%s\", \"pm\":\"%s\", \"med\":\"%s\", \"",
        "\"time\":\"%s\", \"rtc\":\"%s\", \"awaiting\":\"%s\", \"",
        "\"event_id\":\"%u\", \"last_event\":\"%s\"}",
        amStr, pmStr, medName,
        rtcTimeString().c_str(),
        rtcIsRunning() ? "true" : "false",
        awaitingConfirm ? "true" : "false",
        (unsigned)eventId,
        lastEventJson);
    server.send(200, "application/json", buf);
}
```

On remarque que `last_event` est inséré tel quel (%s) : c'est une chaîne JSON déjà formée, conservée dans la variable `lastEventJson`. Cette astuce évite d'allouer un document JSON imbriqué sur un microcontrôleur où la mémoire est comptée.

Chapitre 5

Le modèle d'événements : « pousser » sans push

5.1 Le problème et sa solution

Comme l'appareil ne peut pas joindre le téléphone, comment l'application apprend-elle qu'une pilule vient d'être distribuée, ou que l'utilisateur a appuyé sur le bouton ?

La réponse tient en deux variables côté firmware :

```
static uint8_t eventId          = 0;      // compteur monotone
static char    lastEventJson[80] = "null"; // Le dernier événement, en JSON
```

À chaque fois qu'un événement matériel se produit, l'appareil appelle `pushEvent()`, qui **incrémente le compteur** et **mémore** la description de l'événement :

```
static void pushEvent(const char* json) {
    eventId++;
    strncpy(lastEventJson, json, sizeof(lastEventJson) - 1);
    lastEventJson[sizeof(lastEventJson) - 1] = '\0';
}
```

Ces deux valeurs sont renvoyées dans chaque réponse `/status`. L'application n'a donc qu'à **surveiller `event_id`** : tant qu'il ne change pas, rien de neuf; dès qu'il augmente, il y a un nouvel événement à traiter, décrit par `last_event`.

5.2 Pourquoi un compteur plutôt qu'une file

Ce mécanisme à un seul créneau (« le dernier événement ») est volontairement simple. Comme l'application interroge toutes les 3 secondes et que les événements matériels sont rares (deux distributions par jour, plus les confirmations), il n'y a pratiquement aucun risque d'en perdre un. Le compteur ga-

rantit en plus que **chaque événement n'est traité qu'une seule fois**, sans doublon : l'application retient le dernier event_id vu et ne réagit que lorsqu'il diffère.

5.3 Les types d'événements

Trois familles d'événements peuvent apparaître dans last_event :

evt	type possible	Quand il survient
dispensed	AM, PM	L'heure programmée est atteinte, le moteur a tourné
dispensed	TEST	Distribution déclenchée manuellement via POST /motor
confirmed	AM, PM, TS	L'utilisateur a appuyé sur le bouton physique
error	—	Défaut matériel; un champ msg décrit la cause

Côté application, ces formes sont décrites par un type TypeScript qui sert de contrat :

```
export type HardwareEvent =
  | { evt: 'dispensed'; type: 'AM' | 'PM' | 'TEST' }
  | { evt: 'confirmed'; type: 'AM' | 'PM' | 'TS' }
  | { evt: 'error'; msg: string };
```

Chapitre 6

Écriture vers l'appareil : les routes POST

Toutes les écritures suivent le même schéma : l'application envoie un corps JSON, l'appareil le decode avec `ArduinoJson`, agit, puis répond 200 `{"ok":true}` en cas de succès ou 400 `{"ok":false,"error":"bad JSON"}` si le corps est illisible. Les deux fonctions d'aide centralisent ces réponses :

```
static void ok() { cors(); server.send(200, "application/json", "{\"ok\":true}"); }
static void bad() { cors(); server.send(400, "application/json",
                                     "{\"ok\":false,\"error\":\"bad JSON\"}"); }
```

6.1 /sync — pousser l'horaire et le médicament

C'est par cette route que l'application transmet la configuration choisie par l'utilisateur. Corps attendu :

```
{ "am": "08:00", "pm": "20:00", "med": "Metformin" }
```

Le firmware décompose les chaînes HH:MM en heures et minutes, met à jour le nom du médicament, **enregistre le tout en EEPROM** (pour survivre à une coupure) et rafraîchit l'écran :

```
static void handleSync() {
    JsonDocument doc;
    if (deserializeJson(doc, server.arg("plain")) != DeserializationError::Ok) { bad(); return; }

    const char* am = doc["am"];
    const char* pm = doc["pm"];
    const char* med = doc["med"];

    if (am && strlen(am) == 5) {
        amHour = (uint8_t)((am[0]-'0')*10 + (am[1]-'0'));
        amMin = (uint8_t)((am[3]-'0')*10 + (am[4]-'0'));
    }
}
```

```

}
if (pm && strlen(pm) == 5) { /* idem pour pm */ }
if (med) { strncpy(medName, med, 50); medName[50] = '\0'; }

saveSchedule();           // écriture EEPROM
lcdShowSchedule(amStr, pmStr); // mise à jour de l'écran
ok();
}

```

Côté application, l'appel est trivial :

```

const syncSchedule = useCallback(async (s: ScheduleLike) => {
  await httpService.post('sync', { am: s.am_time, pm: s.pm_time, med: s.medication_name });
}, []);

```

6.2 /settime — régler l'horloge

L'appareil de cette version n'a pas d'horloge sauvegardée par pile : à chaque connexion, l'application lui pousse l'heure du téléphone pour que les distributions se déclenchent à la bonne heure locale. Corps attendu :

```
{ "h": 14, "m": 30, "s": 0, "day": 16, "month": 6, "year": 2026 }
```

Le firmware applique l'heure et réinitialise le garde de distribution pour que la dose du jour puisse se déclencher au bon moment :

```

static void handleSetTime() {
  JsonDocument doc;
  if (deserializeJson(doc, server.arg("plain")) != DeserializationError::Ok) { bad(); return; }
  rtcSetTime(doc["year"] | 2026, doc["month"] | 1, doc["day"] | 1,
             doc["h"] | 0, doc["m"] | 0, doc["s"] | 0);
  lastDispensedMin = -1; // autorise la dose du jour à se redéclencher
  ok();
}

```

6.3 /motor — faire tourner le distributeur

Utilisée pour un test manuel depuis l'application. Corps : { "slots": 1 }. L'appareil fait tourner le moteur du nombre de crans demandé, affiche l'opération, puis publie un événement dispensed de type TEST :

```

static void handleMotor() {
  JsonDocument doc;

```

```

    if (deserializeJson(doc, server.arg("plain")) != DeserializationError::Ok) { bad(); return; }
    int slots = doc["slots"] | 1;
    lcdShowDispensing();
    motorRotate(slots);
    lcdShowDone();
    pushEvent("{\"evt\":\"dispensed\",\"type\":\"TEST\"}");
    awaitingConfirm = true;
    ledBlinking = true;
    ok();
}

```

Un cran correspond à **2048 demi-pas**, soit une demi-révolution du moteur 28BYJ-48, ce qui avance le carrousel d'exactly un compartiment.

6.4 /led — piloter la LED indicatrice

Corps: { "state": 1 } pour allumer, { "state": 0 } pour éteindre. L'écriture manuelle interrompt tout clignotement automatique en cours :

```

static void handleLed() {
    JsonDocument doc;
    if (deserializeJson(doc, server.arg("plain")) != DeserializationError::Ok) { bad(); return; }
    ledBlinking = false;
    ledState = doc["state"] | 0;
    digitalWrite(LED_PIN, ledState ? HIGH : LOW);
    ok();
}

```

6.5 /diag et /selftest — diagnostics

/diag (GET) scanne le bus I2C et renvoie un état de santé complet : présence du RTC, présence de l'écran, adresses I2C détectées, mémoire libre et puissance du signal WiFi. /selftest (GET) déclenche un test lent du moteur pas-à-pas. Ces deux routes permettent à l'application de **sonder le matériel sans lancer un vrai cycle de distribution**.

Chapitre 7

Le client HTTP côté application

Toute la communication côté téléphone est concentrée dans une seule classe singleton, `HttpService`. Comprendre cette classe, c'est comprendre tout le comportement réseau de l'application.

7.1 La gestion de l'adresse

L'application accepte une IP nue ou une IP avec port, et la normalise en URL :

```
setUrl(ip: string): void {  
    const clean = ip.trim().replace(/^https?:\\\/\\\/, '');  
    this.baseUrl = `http://${clean}`;  
}
```

L'adresse est ensuite persistée en base SQLite (`schedule.device_ip`) afin d'être pré-remplie au prochain lancement de l'application.

7.2 Le cycle de connexion, étape par étape

La méthode `connect()` exécute une séquence fixe à chaque tentative :

```
async connect(): Promise<void> {  
    if (!this.baseUrl) throw new Error('No device IP set. Enter the IP shown on the LCD.');
```

// 1. Vérifier que l'appareil répond

```
    try {  
        const resp = await fetch(`${this.baseUrl}/status`, { signal: withTimeout(TIMEOUT_MS) });  
        if (!resp.ok) throw new Error('Device returned an error response.');
```

} catch (e: any) {

```
        this.emitStatus('disconnected');
```

```

    /* messages d'erreur adaptés selon AbortError ou non */
    throw new Error('Could not reach the device. Check the IP address and your WiFi network');
}

// 2. Synchroniser l'heure du téléphone vers l'appareil
const now = new Date();
await fetch(`${this.baseUrl}/settime`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    h: now.getHours(), m: now.getMinutes(), s: now.getSeconds(),
    day: now.getDate(), month: now.getMonth() + 1, year: now.getFullYear(),
  }),
  signal: withTimeout(TIMEOUT_MS),
}).catch(() => { /* non bloquant */ });

// 3. Démarrer l'interrogation périodique
this.emitStatus('connected');
this.startPolling();
}

```

En clair, la connexion enchaîne : (1) un **ping** sur /status qui sert de test de joignabilité; (2) une **synchronisation d'horloge** via /settime; (3) le **démarrage du polling**. Si le ping échoue, l'état repasse immédiatement à « déconnecté » et un message d'erreur explicite est remonté à l'utilisateur.

7.3 La boucle d'interrogation

Le cœur de la détection d'événements est la boucle lancée par startPolling() :

```

private startPolling(): void {
  this.pollTimer = setInterval(async () => {
    if (!this.baseUrl) return;
    try {
      const resp = await fetch(`${this.baseUrl}/status`, { signal: withTimeout(TIMEOUT_MS) });
      if (!resp.ok) throw new Error('bad status');
      const data = await resp.json();

      if (typeof data.event_id === 'number' && data.event_id !== this.lastEventId) {
        this.lastEventId = data.event_id;
        if (data.last_event && data.last_event !== null) {
          this.eventHandlers.forEach((h) => h(data.last_event as HardwareEvent));
        }
      }
    }
  });
}

```

```

    }
  } catch {
    this.stopPolling();
    this.emitStatus('disconnected');
  }
}, POLL_MS);
}

```

Trois constantes gouvernent ce comportement :

- `POLL_MS = 3000` — l'état est redemandé toutes les **3 secondes**;
- `TIMEOUT_MS = 6000` — chaque requête abandonne après **6 secondes**;
- la comparaison `data.event_id !== this.lastEventId` garantit qu'un événement n'est livré **qu'une fois** aux gestionnaires abonnés.

Si une seule interrogation échoue (timeout, perte de WiFi...), la boucle s'arrête et l'application bascule automatiquement en « déconnecté » — la perte de lien est donc détectée en quelques secondes, sans intervention de l'utilisateur.

7.4 L'envoi des commandes

Toutes les écritures passent par une méthode `post()` unique, qui applique le même en-tête, le même délai et la même gestion d'erreur :

```

async post(endpoint: string, body: object = {}): Promise<void> {
  if (!this.baseUrl) throw new Error('Not connected');
  const resp = await fetch(`${this.baseUrl}/${endpoint}`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(body),
    signal: withTimeout(TIMEOUT_MS),
  });
  if (!resp.ok) throw new Error(`Device rejected the command (HTTP ${resp.status})`);
}

```

Le contexte React (`HardwareContext`) habille ces appels en fonctions métier lisibles — `syncSchedule`, `testLed`, `testMotor` — et expose l'état de connexion et le dernier événement à toute l'interface.

Chapitre 8

Scénarios de bout en bout

8.1 Scénario A — la dose programmée

C'est le scénario nominal, entièrement piloté par l'appareil ; l'application ne fait qu'observer.

1. **Appareil.** Dans `loop()`, la fonction `checkSchedule()` compare l'heure courante aux heures AM/PM. Quand elles coïncident, elle appelle `dispense("AM")`.
2. **Appareil.** `dispense()` affiche « Distribution », fait tourner le moteur d'un cran (`motorRotate(1)`), affiche « Terminé », met la LED en clignotement, passe `awaitingConfirm` à `true`, puis publie l'événement :

```
static void dispense(const char* type) {
    lcdShowDispensing();
    motorRotate(1);
    lcdShowDone();
    ledBlinking      = true;
    awaitingConfirm = true;
    char evt[64];
    snprintf(evt, sizeof(evt), "{\"evt\":\"dispensed\",\"type\":\"%s\"}", type);
    pushEvent(evt);    // event_id++
}
```

3. **Application.** Lors de la prochaine interrogation (au plus 3 s plus tard), elle voit un nouvel `event_id` et déclenche l'affichage « Dose libérée — veuillez confirmer ».
4. **Utilisateur.** Il prend la pilule et appuie sur le bouton physique.
5. **Appareil.** `checkButton()` déboune l'appui, allume la LED en continu 1,5 s, réinitialise `awaitingConfirm`, et publie un événement `confirmed`.
6. **Application.** La prochaine interrogation voit encore un nouvel `event_id` et **enregistre automatiquement la dose comme « Prise »**, avec une alerte à l'utilisateur.

La garde anti-double-distribution mérite d'être notée : `lastDispensedMin` empêche la même dose de se déclencher deux fois dans la même minute, et est remis à zéro à minuit pour le lendemain.

```
static void checkSchedule() {
    if (!rtcIsRunning()) return;
    uint16_t nowMins = rtcHour() * 60 + rtcMinute();
    uint16_t amMins = amHour * 60 + amMin;
    uint16_t pmMins = pmHour * 60 + pmMin;
    if (nowMins == amMins && lastDispensedMin != (int)amMins) {
        lastDispensedMin = amMins; dispense("AM");
    } else if (nowMins == pmMins && lastDispensedMin != (int)pmMins) {
        lastDispensedMin = pmMins; dispense("PM");
    }
    if (nowMins == 0 && lastDispensedMin > 0) lastDispensedMin = -1; // reset minuit
}
```

8.2 Scénario B — la première connexion

1. L'utilisateur met l'appareil sous tension; l'écran affiche l'IP obtenue (ex. 192.168.1.5) pendant 5 secondes.
2. Dans l'application, onglet **Appareil**, il saisit cette IP et appuie sur **Connecter**.
3. L'application ping /status, pousse l'heure via /settime, démarre le polling, et l'indicateur passe au vert. L'IP est mémorisée.
4. Au prochain lancement, l'IP est pré-remplie : un seul appui sur **Connecter** suffit.

8.3 Scénario C — la modification de l'horaire

1. L'utilisateur change l'heure du matin dans les réglages et enregistre.
2. L'application appelle `syncSchedule()`, qui poste sur /sync.
3. L'appareil met à jour ses variables, écrit en EEPROM, et **rafraîchit l'écran en moins de 2 secondes**. Le nouvel horaire survivra désormais à toute coupure.

8.4 Scénario D — la perte de réseau

1. Le téléphone quitte le WiFi (ou l'appareil s'éteint).
2. La prochaine interrogation expire au bout de 6 s; la boucle s'arrête et l'état passe à « déconnecté ».
3. De son côté, l'appareil continue de distribuer en autonomie : aucune dose n'est manquée du fait de l'absence de l'application.
4. Quand le WiFi revient, l'utilisateur (ou la reconnexion) relance `connect()`.

Chapitre 9

Robustesse et détails d'implémentation

Plusieurs garde-fous, parfois discrets, rendent la communication fiable :

- **Délais systématiques.** Chaque requête côté application est munie d'un `AbortSignal` de 6 s. Un appareil bloqué ne fige donc jamais l'interface.
- **Itinérance multi-WiFi.** Le firmware connaît plusieurs réseaux via `ESP8266WiFiMulti` et rejoint automatiquement le meilleur disponible. En cas de coupure, `maintainWifi()` retente la connexion environ toutes les 8 secondes.
- **Persistance EEPROM.** L'horaire et le nom du médicament sont stockés avec un octet magique `0xAB` qui valide le bloc. Au démarrage, `loadSchedule()` ne recharge les valeurs que si cet octet est présent — sinon les valeurs par défaut s'appliquent.
- **Distribution idempotente.** Le garde `lastDispensedMin` évite tout doublon dans la même minute (cf. scénario A).
- **CORS.** Chaque réponse porte les en-têtes `Access-Control-Allow-*`, et chaque route `POST` répond aux requêtes préliminaires `OPTIONS` :

```
static void cors() {  
    server.sendHeader("Access-Control-Allow-Origin", "*");  
    server.sendHeader("Access-Control-Allow-Methods", "GET, POST, OPTIONS");  
    server.sendHeader("Access-Control-Allow-Headers", "Content-Type");  
}
```

- **Démarrage prudent du moteur.** Le test moteur n'est volontairement **pas** lancé au démarrage : le pic de courant du moteur pourrait faire chuter la tension et empêcher l'ESP8266 d'atteindre le WiFi. Il est donc déporté sur la route `/selftest`, à la demande.

Chapitre 10

Référence rapide

10.1 Tableau récapitulatif des échanges

Sens	Déclencheur	Endpoint	Corps / Réponse
App → Appareil	Connexion	GET /status	— / état complet
App → Appareil	Connexion	POST /settime	heure du téléphone / { "ok":true }
App → Appareil	Réglages enregistrés	POST /sync	{ am,pm,med } / { "ok":true }
App → Appareil	Test LED	POST /led	{ "state":1 } / { "ok":true }
App → Appareil	Test moteur	POST /motor	{ "slots":1 } / { "ok":true }
App → Appareil	Toutes les 3 s	GET /status	— / état + event_id
Appareil → App	Dose distribuée	(via /status)	last_event = { evt:"dispensed", type:"AM" }
Appareil → App	Bouton pressé	(via /status)	last_event = { evt:"confirmed", type:"AM" }

10.2 Constantes clés

Constante	Valeur	Où
Port HTTP	80	firmware (HTTP_PORT)
Intervalle de polling	3000 ms	app (POLL_MS)
Délai de requête	6000 ms	app (TIMEOUT_MS)

Constante	Valeur	Où
Demi-pas par cran	2048	firmware (STEPS_PER_SLOT)
Octet magique EEPROM	0xAB	firmware (EEPROM_MAGIC)

10.3 Conclusion

Le modèle se résume en cinq affirmations qui structurent tout le système :

1. **Transport** — uniquement du HTTP transportant du JSON, sur le WiFi local.
2. **Rôles** — le distributeur est le serveur (port 80), l'application est l'unique client.
3. **App → Appareil** — des écritures REST : /sync, /settime, /led, /motor.
4. **Appareil → App** — un /status interrogé toutes les 3 s, porteur d'un event_id croissant qui livre les événements dispensed, confirmed et error.
5. **Autonomie** — l'appareil distribue et confirme seul ; l'application se contente de le configurer et de refléter fidèlement ce qui s'est produit.

Le téléphone configure. L'appareil agit. Le statut réconcilie.